EECS 484, Winter 2026 - Database Management Systems

# DBMS Overview

## Chapter 1

Instructor: Barzan Mozafari

University of Michigan, Ann Arbor

# What is a DBMS?

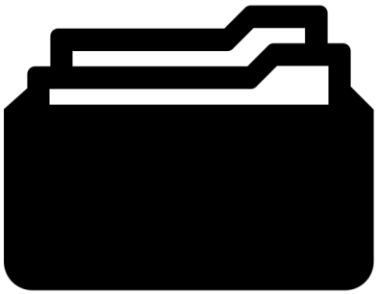
- Database: Large, integrated collection of data
- DBMS = Database Management System
  = a software package designed to store and manage databases
- Examples:
  - Oracle, MS SQL server, DB2, MySQL, Postgres
- **Goal**: Provide an abstraction layer for storing and accessing data
  - Why abstraction? For efficiency and convenience

# DBMS: Motivation

- Consider the following scenario:
  - Students taking classes, and obtaining grades
  - Q1: Find grade in class X
  - Q2: Find my GPA
  - …

# Old-time Solution: Sorted Student Folders

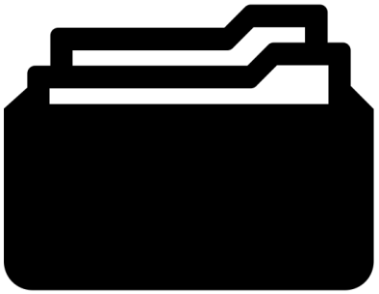
- Advantages?
- Disadvantages?



## Ideas?

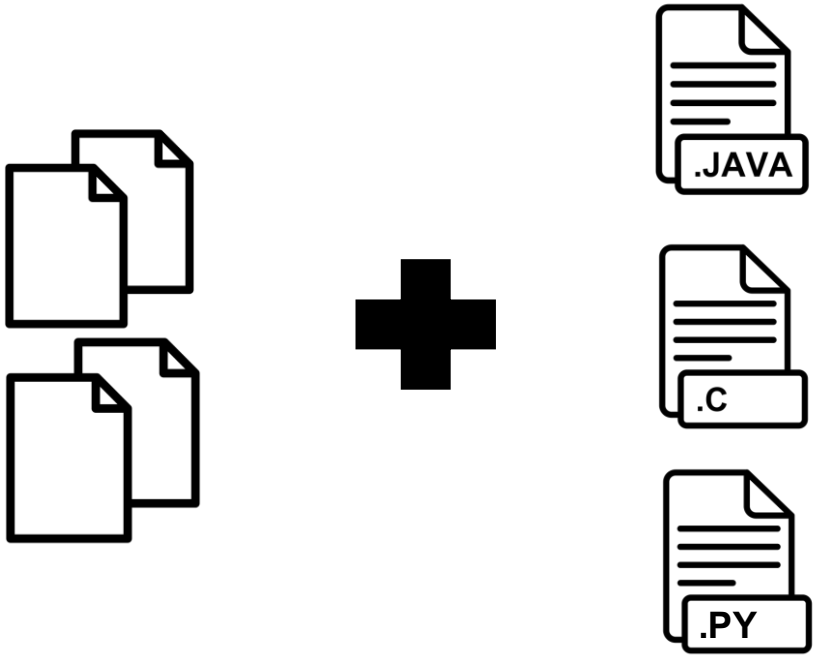
# Answer

- Advantages?
  - Cheap
- Disadvantages?
  - Large physical footprint
  - No sharing
  - No ad-hoc queries



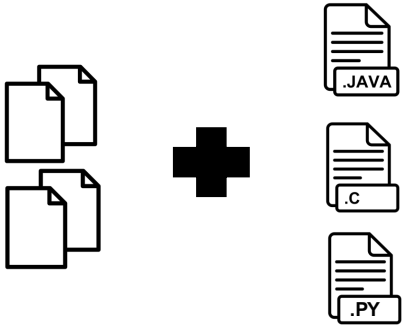
# Other Solution: Flat Files

- Access?
  - using programs in C, Java, Python etc.
- Layout for the student records?



# Other Solution: Flat Files

- Access?
  - using programs in C, Java, etc.
- Layout for the student records?

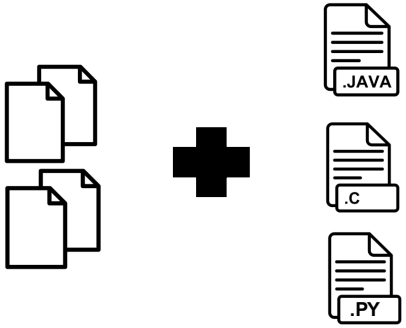



**CSV:**

Brown, Lisa, lbrown, 484, A+, 482, D
Smith, Bart, bsmith
Tompson, Mary, mtom, 281, B+, 484, A
…
…

# Other Solution: Flat Files

- Access?
  - using programs in C, Java, etc.
- Layout for the student records?



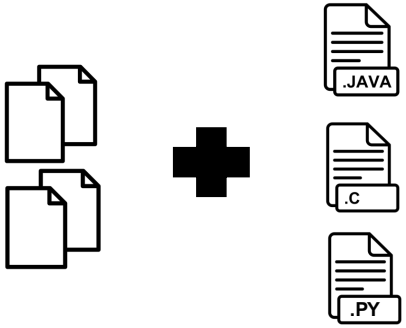

**Multiple files:**

Brown, Lisa, lbrown
Smith, Bart, bsmith
Tompson, Mary, mtom
…
…

lbrown, 484, A+
lbrown, 482, D
mtom, 281, B+
mtom, 484, A
…
…

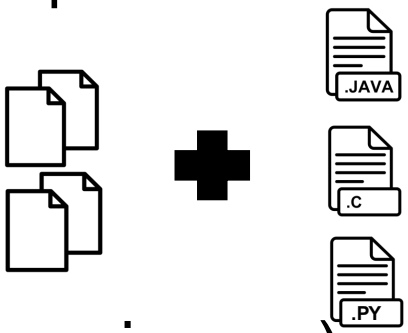
# Other Solution: Flat Files

- Problems?



# Answer

- Problems?
  - Inconvenient access to data
    - requires programming experience and knowledge of file layout
    - Any changes to the format of data requires reprogramming
  - Data redundancy
  - Integrity problems
  - Atomicity problems (concurrent access issues)
  - Security problems



# Why use a DBMS?

- It solves ALL problems with file-based data mngt.

# But How?

# Why use a DBMS?

- It solves ALL problems with file-based data mngt.
  - Reduces application dev time
  - Allows for efficient storage and access
  - Shields apps from changes to internal representation and storage format
  - Protects data integrity and security
  - Enables concurrent access & recovery from crashes

**But HOW???**

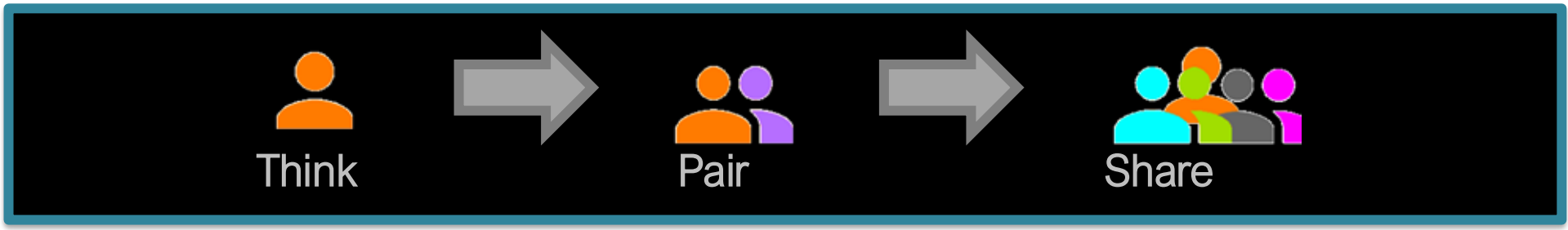
# Declarative vs. Imperative Languages

- Declarative: Express **what** your goal is
  - Return all students who have a GPA higher than their last year's GPA

- Imperative: Express **how** to achieve your goal
  - Read this file,
  - For each record, add up their grades until last yr
  - Record last year's GPA in an array
  - ...

# CYU: Declarative Interfaces

- Pros?
- Cons?



# Answer

- Pros?
  - Faster to write
  - Easier to debug
  - Allows the computer to choose the best execution plan
- Cons?
  - Can be expensive to find the best plan
  - May not find the best plan => slower than manually optimized code (imperative langs)
  - Typically, less expressive

# Why use a DBMS?

- It solves ALL problems with file-based data mngt.
  - Declarative interface
    - Reduces application dev time
    - Allows for efficient storage and access
  - Shields apps from changes to internal representation and storage format
  - Protects data integrity and security
  - Enables concurrent access & recovery from crashes

# Relational and Other Data Models

- **DBMS using the relational DM** ('70s-now)
  - IBM DB2
  - Informix
  - Oracle
  - Sybase
  - Microsoft Access
  - Tandem
  - Teradata
  - …
- **Other data models**
  - Hierarchical (mid '60s-'70s)
    - IBM IMS
  - Network ('70s)
    - IDMS, IDS
  - Object-oriented (~'90s)
    - ObjectStore
  - Object-relational (relational model + object DB concepts)
    - Oracle
  - …

# Relational (Data) Model

- The most widely-used model today
- **Data model** = a collection of concepts for describing data
  - A collection of relations
  - Relation = set of records – think of it as a table with rows and columns

Students

| sid | name | login | age |
|---|---|---|---|
| 13 | Lisa | lsimp | 40 |
| 41 | Bart | bart | 20 |

Courses

| cid | cname | cred. |
|---|---|---|
| E-484 | EECS484 | 4 |
| E-482 | EECS584 | 4 |

Enrolled

| sid | cid | grade |
|---|---|---|
| 41 | E-484 | A+ |
| 13 | E-482 | C- |

# Relational (Data) Model

- **Schema** = a description of data in terms of a data model
  - Every relation has a schema
  - Specifies the name of the relation, the name and type of the columns (or fields or attributes)
  - Each row also called a tuple or a record

Students(sid:`string`, name:`string`, login:`string`, age:`integer`)
Courses(cid:`string`, cname:`string`, credits:`integer`)
Enrolled(sid:`string`, cid:`string`, grade:`string`)

Students

| sid | name | login | age |
|---|---|---|---|
| 13 | Lisa | lsimp | 40 |
| 41 | Bart | bart | 20 |

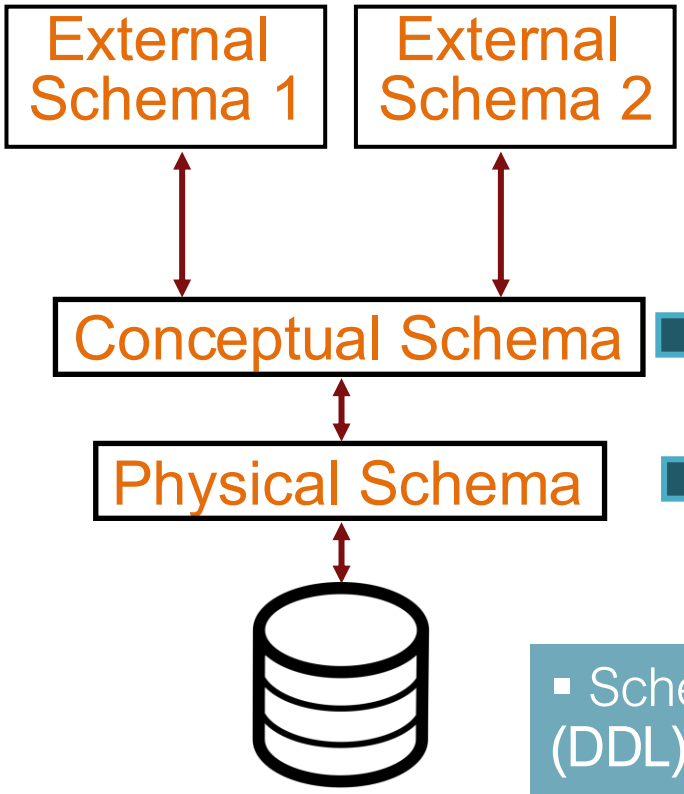
Courses

| cid | cname | cred. |
|---|---|---|
| E-484 | EECS484 | 4 |
| E-584 | EECS584 | 3 |

Enrolled

| sid | cid | grade |
|---|---|---|
| 41 | E-484 | A- |
| 13 | E-584 | A+ |

# Levels of Abstraction



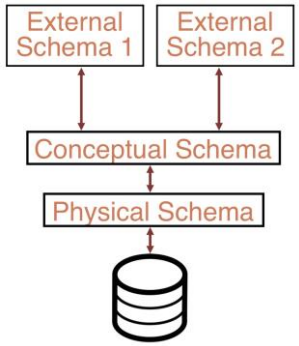

describe how users see the data, views

defines the logical structure in terms of the DBMS's data model

describes the files and indexes used

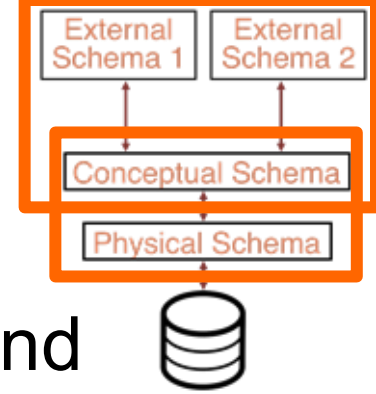
- Schemas are defined using **Data Definition Language (DDL) – e.g.,** `CREATE TABLE X (…)`
- Data is modified/queried using **Data Manipulation Language (DML) – e.g.,** `SELECT FROM X WHERE …`

# Example




- Conceptual schema (1):
  - Students(sid:`string`, name:`string`, login:`string`, age:`integer`)
  - Courses(cid:`string`, cname:`string`, credits:`integer`)
  - Enrolled(sid:`string`, cid:`string`, grade:`string`)
- Physical schema (1):
  - Relations stored as unordered files.
  - Index on first column of Students.
- External Schema (≥ 1):
  - View: Course_info(cid:`string`, enrollment:`integer`)
  - View: Class_rank(sid:`string`, gpa:`real`, rank:`integer`)

# Data Independence




- Applications insulated from data format and storage details
- Logical data independence: Protection from changes in *logical* structure of data
  - External schemas
- Physical data independence: Protection from changes in *physical* structure of data
  - Conceptual schemas

# Example: Why Separate Schemas Allow for Data Independence

## students_v1.csv

**uniqname, first, last, major**
sjohnson, Sarah, Johnson, CSE
mmiller, Mary, Miller, Math
jdoe, John, Doe, Econ

## C++: print uniqname and major.

```
#include <fstream>
#include <iostream>
#include <sstream>
using namespace std;

int main() {
  ifstream in("students_v1.csv");
  string line,u,f,l,m;
  getline(in,line);                    // header
  while (getline(in,line)) {
    stringstream ss(line);
    getline(ss,u,','); getline(ss,f,','); getline(ss,l,','); getline(ss,m,',');
    cout << u << " -> " << m << "\n";
  }
}
```

## Output:

**sjohnson, CSE**
**Mmiller, Math**
**jdoe, Econ**

## SQL: print uniqname and major.

SELECT uniqname, major
FROM Students;

# Example: Why Separate Schemas Allow for Data Independence

## students_v2.csv

**uniqname, sport, first, last, major**
sjohnson, soccer, Sarah, Johnson, CSE
mmiller, tennis, Mary, Miller, Math
jdoe, wrestling, John, Doe, Econ

## C++: print uniqname and major.

```
#include <fstream>
#include <iostream>
#include <sstream>
using namespace std;

int main() {
  ifstream in("students_v1.csv");
  string line,u,f,l,m;
  getline(in,line);                    // header
  while (getline(in,line)) {
    stringstream ss(line);
    getline(ss,u,','); getline(ss,f,','); getline(ss,l,','); getline(ss,m,',');
    cout << u << " -> " << m << "\n";
  }
}
```

## Output:

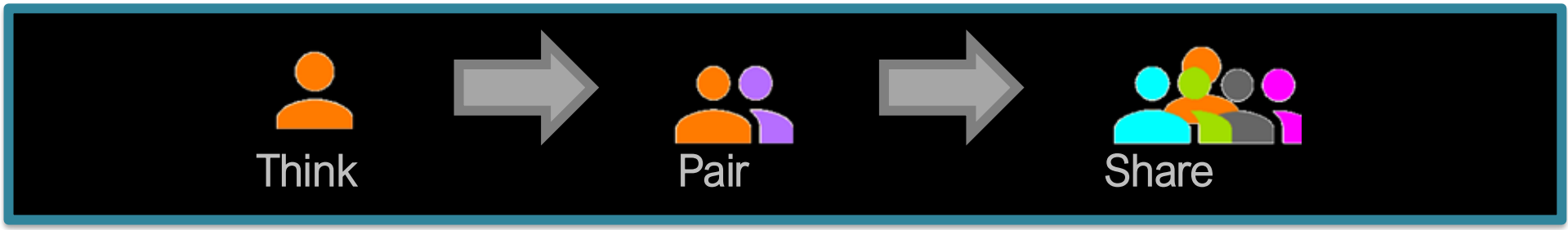
**sjohnson, CSE**
**Mmiller, Math**
**jdoe, Econ**

## SQL: print uniqname and major.

SELECT uniqname, major
FROM Students;

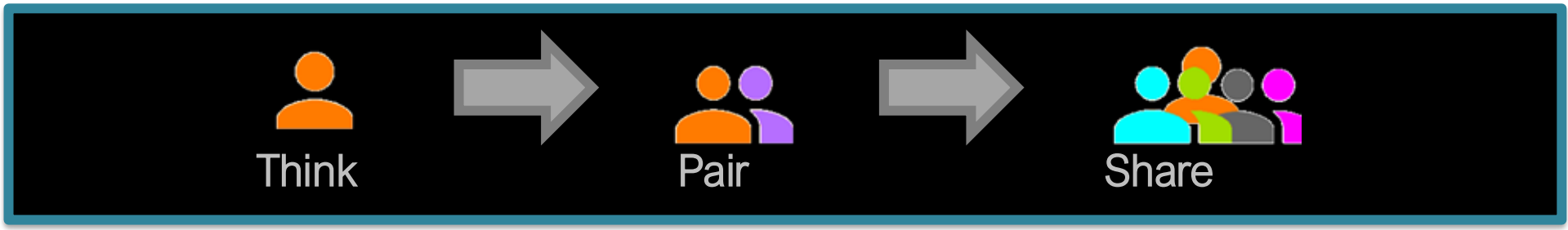
# CYU

- Which of these are more suitable for storing in a DBMS rather than files in an OS?
  - (a) Grades for students at the university
  - (b) Source code for a program
  - (c) Contents of a textbook



# CYU

- While UM uses a single relational table with information about both your grades and number of credits, Canvas only allows you to see your letter grades. Does Canvas represent:
  a) An external schema?
  b) A conceptual schema?
  c) A physical schema?



# Why use a DBMS?

- It solves ALL problems with file-based data mngt.
  - Declarative interface
    - Reduces application dev time
    - Allows for efficient storage and access
  - Data independence
    - Shields apps from changes to internal representation and storage format
  - Protects data integrity and security
  - Enables concurrent access & recovery from crashes

# Why use a DBMS?

- It solves ALL problems with file-based data mngt.
  - Declarative interface
    - Reduces application dev time
    - Allows for efficient storage and access
  - Data independence
    - Shields apps from changes to internal representation and storage format
  - Centralized data administration
    - Protects data integrity and security
  - Enables concurrent access & recovery from crashes

# Why use a DBMS?

- It solves ALL problems with file-based data mngt.
  - Declarative interface
    - Reduces application dev time
    - Allows for efficient storage and access
  - Data independence
    - Shields apps from changes to internal representation and storage format
  - Centralized data administration
    - Protects data integrity and security
  - Transaction support
    - Enables concurrent access & recovery from crashes

# Who uses a DBMS?

- Everyone!
  - Your bank
  - Your university
  - Your coffee shop
  - Your favorite hotel
  - Your favorite website
  - Your phone
  - Your government
- How many databases have you used so far today?

amazon

canvas

# Database Administrator (DBA)

- ~~Highly paid~~ people in charge of running a DBMSs
  - Designs logical /physical schemas
  - Handles security and authorization
  - Data availability, crash recovery
  - Database tuning as needs evolve

- With the rise of cloud DBs (DB-as-a-Service), the DBA role is eliminated or at least changing drastically in many organizations

Note: Modern DBAs are often called **DBRE (Database Reliability Engineer)** or **Database Platform Engineer**, esp. in DevOps-style orgs

# Summary

- DBMS used to maintain & query large datasets
- Benefits include
  - Faster app dev (via declarative lang) & easier maintenance (via data independence)
  - recovery from system crashes
  - concurrent access
  - data integrity and
  - data sharing & security
- Levels of abstraction give data independence
- ~~DBAs hold interesting, well-paid jobs~~

Optional

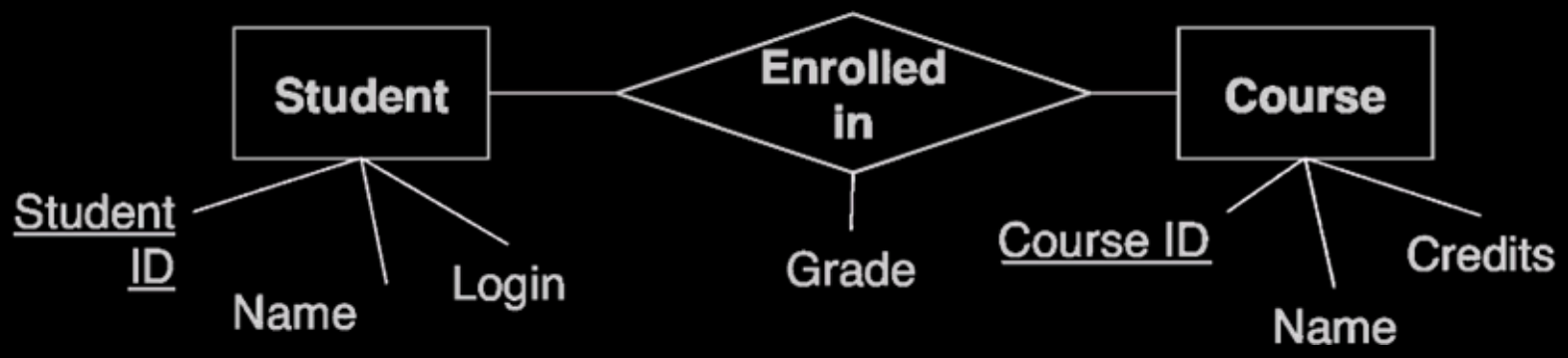
# DBAs vs Data Engineers

- **DBA's focus**
  - Keeping DBs healthy & safe
  - Examples
    - Reliability & uptime
    - Backups/restore,
    - Replication, capacity planning
    - Performance tuning
    - Security
    - Upgrades/patching (less so on fully managed cloud, but still planning/testing)
- **DE's focus**
  - Moving/transforming data so it's usable for analytics/ML/apps
  - Examples
    - Building pipelines (ETL/ELT)
    - Ingestion from many sources
    - Data modeling for analytics (warehouse/lakehouse layers, dimensions/facts, marts)
    - Orchestration & reliability of pipelines (scheduling, retries, lineage)
    - Data quality testing/monitoring, SLAs for datasets
    - Serving data to BI/ML/features

Optional

# How DBA's Job is Changing

- Many classic DBA chores are getting **automated** by cloud
- **Companies still need DB expertise**, but for
  - *architecture choices (sharding/partitioning, consistency tradeoffs)*
  - *cost/performance tradeoffs*
  - *security & compliance*
  - *query/index design support for teams*
  - *reliability engineering (SLOs, observability, incident playbooks)*

# Next

- The Entity-Relationship (ER) Model